Compound Error in the Multimodal PAD Pipeline

What compound error is (general case)

Compound error is when small errors at each stage of a pipeline multiply, so the end result is much worse than any single model's accuracy suggests.

If models run in sequence (A feeds B feeds C), the system's accuracy is roughly the product of each stage's accuracy:

- 3 models at 90% each: 0.9 × 0.9 × 0.9 ≈ **73%**
- 5 models at 90% each: ≈ **59%**
- 5 models at 95% each: ≈ **77%**

Each model looks good on its own, but the chain is only as reliable as the product of all links. It is often worse than the math suggests, for two reasons:

1. **Distribution shift.** Model B was trained on clean, ground-truth inputs, but in production it receives Model A's imperfect outputs. B has never seen those kinds of mistakes, so it can fail in unexpected ways.
2. **Error amplification.** Some downstream models take a small upstream mistake and make it much bigger (a slightly wrong crop leads to a totally wrong classification).

How to reduce it (general case)

1. Measure end-to-end, not per model. Evaluate the whole pipeline on a real test set, and also log each stage's error so you know which link is the weakest. Fix the worst one first.

2. Train downstream models on upstream outputs. Instead of feeding B ground-truth inputs during training, feed it what A actually produces, including its mistakes. This closes the train/test mismatch and is usually the biggest single win.

3. Pass soft information, not hard decisions. Instead of sending A's final label, pass probabilities, embeddings, or confidence scores. A hard "cat" throws away the fact that A was only 51% sure.

4. Add confidence thresholds and fallbacks. If A is unsure, don't pass the result on blindly: route it to a stronger model, a human, or return "uncertain".

5. Fine-tune jointly (end-to-end). If the models are differentiable, train the whole chain together so B learns to compensate for A's errors, and A learns what B actually needs.

6. Reduce the number of stages. Every extra model is another multiplication. Ask whether one model could do two jobs, or whether a stage can be replaced by simple rules.

7. Run models in parallel instead of in series. Ensembles (voting, averaging, stacking) can reduce error because independent mistakes cancel out, rather than compounding.

8. Add validation between stages. Cheap sanity checks (format, range, consistency) can catch obvious garbage before it propagates.

9. Use iterative correction. Let a later stage send feedback to an earlier one (re-run with a different crop, a different prompt, etc.) instead of a strictly one-way flow.

Quick diagnostic (general case)

Take a set of examples where the pipeline fails and test: if I replace A's output with the correct answer, does B succeed? If yes, the problem is upstream error propagation. If B still fails, B itself is the problem. This tells you where to spend your effort.

Where the compound error lives

The pipeline is two parallel branches that fuse at the end, with one serial chain inside the bottom branch. Compound error only hits one part of it.

Top branch (audio): Wav2Vec2 → PAD head. This is one model, trained end-to-end. No compounding.

Bottom branch (text): Speech-to-text (Parakeet) → Text-based PAD regression. This is the risky chain:

- ASR errors (wrong words, dropped words, mishearing in noisy game audio) go straight into the text regressor.
- The regressor was probably trained on clean text, but now receives noisy transcripts. This is the distribution shift problem.
- Game voice chat makes it worse: background noise, overlapping speakers, slang, shouting, and emotional speech are exactly where ASR is weakest, and those are also the moments that matter most for PAD (pleasure, arousal, dominance).

Fusion: it combines two predictions. If it is naive (fixed averaging), a bad text prediction can drag down a good audio one.

What to do, in order of impact

1. Train the text regressor on Parakeet's outputs, not clean transcripts. Run Parakeet on the whole training set and train the PAD regressor on those noisy transcripts. This is the single biggest fix.

2. Pass more than the transcript. Give the regressor ASR confidence scores, word-level confidences, or the n-best hypotheses. Low-confidence words can then be down-weighted instead of trusted.

3. Make fusion confidence-aware. Instead of averaging, learn the weights:

- Gated fusion: a small network decides how much to trust each branch per sample, using ASR confidence, audio quality (SNR), and each branch's own uncertainty.
- Fuse embeddings (Wav2Vec2 features + text embeddings) rather than final PAD values, so the fusion model sees richer information than two numbers.

4. Use modality dropout during training. Randomly zero out one branch so the fusion learns to cope when text or audio is unreliable. This makes the system robust to bad transcripts.

5. Fine-tune Parakeet on your domain, but check the right metric. WER is not what matters. A transcript with 15% WER can still be fine for PAD if the emotional words are right. Evaluate the ASR by how it affects final PAD error, not by WER alone.

6. Fall back to audio when the transcript is bad. If ASR confidence is below a threshold (or the transcript is empty or very short), rely on the audio branch only. Wav2Vec2 hears tone and arousal directly, with no transcription step to fail.

How to diagnose it

Run an ablation on a held-out set:

Setup	What it tells you
Audio only	Baseline for the top branch
Text only (ground-truth transcripts)	Ceiling for the text branch
Text only (Parakeet transcripts)	Real text performance
Fusion	Final result

The gap between ground-truth transcripts and Parakeet transcripts is exactly the compound error. If it is large, fix the ASR-to-regressor link. If it is small and fusion still underperforms, the problem is in the fusion step.

If the fused result is worse than audio-only, that is a strong sign the text branch is adding noise, and confidence-gated fusion or modality dropout should help most.